HG002 Long-Read Variant Calling Pipeline

A Comprehensive Guide to Nextflow,
HPC Bioinformatics, and Variant Calling

Course: Special Topics in Bioinformatics (BI-436)

Program: BS Bioinformatics — 6th Semester

Institution: School of Interdisciplinary Engineering & Sciences (SINES)
National University of Sciences & Technology (NUST), Islamabad

What this document covers:

This guide explains *everything* you need to know to understand, reproduce, and extend a containerized Nextflow pipeline for long-read variant calling on HPC infrastructure. It assumes **zero prior knowledge** of Nextflow, Singularity, SLURM, or variant calling — every concept is built from scratch.

Repository: `https://github.com/cantcode27/HG002-Long-Read-Variant-Calling-Pipeline`

Last updated: March 14, 2026

Contents

1 Introduction **3**

2 Background: The Biology **3**

2.1 DNA and the Human Genome . . . 3

2.2 What is DNA Sequencing? . . . 3

2.3 What is a Reference Genome? . . . 4

2.4 What is Variant Calling? . . . 4

2.5 What is HG002? . . . 4

2.6 The VCF File Format . . . 4

3 Background: The Infrastructure **5**

3.1 What is an HPC Cluster? . . . 5

3.2 SLURM: The Job Scheduler . . . 5

3.2.1 Key SLURM Commands . . . 5

3.2.2 Anatomy of a SLURM Script . . . 6

3.3 Containers: Docker and Singularity . . . 6

3.3.1 Docker vs Singularity . . . 7

4 Nextflow: Complete Beginner's Guide **7**

4.1 What is Nextflow? . . . 7

4.2 DSL2: The Modern Nextflow Syntax . . . 8

4.3 The Two Building Blocks . . . 8

4.3.1 Processes . . . 8

4.3.2 Workflows . . . 9

4.4 Tuples: Passing Multiple Files Together . . . 9

4.5 Parameters and the publishDir Directive . . . 10

4.6 The -resume Flag . . . 10

4.7 Nextflow Configuration File . . . 10

5 The Pipeline: Step-by-Step Explanation **11**

5.1 Pipeline Flow Diagram . . . 11

5.2 Step 1: Reference Indexing (faidx) . . . 11

5.3 Step 2: Read Alignment (minimap2) . . . 11

5.4 Step 3: SAM to BAM Conversion and Sorting . . . 12

5.5 Step 4: Alignment QC (flagstat) 12
5.6 Step 5: Variant Calling 12
5.7 Step 6: Benchmarking with hap.py 12

6 Benchmarking Results and Interpretation 13
6.1 The Raw Numbers 13
6.2 Why is the Recall So Low? 14
6.3 What the Numbers Actually Tell Us 14

7 Practical: Reproducing the Pipeline 14
7.1 Prerequisites 15
7.2 Step-by-Step Commands 15

8 Common Errors and How to Fix Them 16

9 Tools Summary 17

10 Glossary 17

Acknowledgments 18

1 Introduction

This document is a complete, beginner-friendly guide to building and understanding an automated bioinformatics pipeline. By the end, you will understand:

- What variant calling is and why it matters
- How Nextflow works as a workflow manager
- How SLURM schedules jobs on an HPC cluster
- How Singularity containers ensure reproducibility
- How to read, modify, and debug the pipeline code
- How to interpret benchmarking results from hap.py

The pipeline we are studying takes raw PacBio HiFi sequencing reads from the HG002 human genome sample, aligns them to the GRCh38 reference genome, calls variants using two deep-learning callers (Clair3 and DeepVariant), and then benchmarks the results against a gold-standard truth set.

2 Background: The Biology

2.1 DNA and the Human Genome

DNA (deoxyribonucleic acid) is the molecule that stores genetic information in all living organisms. It is made up of four chemical bases — Adenine (A), Thymine (T), Cytosine (C), and Guanine (G). The human genome contains roughly 3.2 billion of these bases arranged in a specific order across 23 pairs of chromosomes.

2.2 What is DNA Sequencing?

DNA sequencing is the laboratory process of reading the order of bases in a DNA molecule. Because the genome is so large, sequencing machines break it into small fragments and read them in parallel. Each fragment that is read is called a **read**.

Short-read sequencing (e.g., Illumina): Produces reads that are roughly 150 bases long. Very accurate per-base, but the short length makes it hard to figure out where repetitive regions belong.
Long-read sequencing (e.g., PacBio HiFi): Produces reads that are 10,000–20,000 bases long. PacBio HiFi reads also have very high accuracy (>99.9%). The long length makes alignment much easier and allows detection of complex structural variants.

2.3 What is a Reference Genome?

A **reference genome** is a representative example of a species' genome. For humans, the current standard is **GRCh38** (also called hg38). It serves as a coordinate system — when we say "a variant at chromosome 1 position 1,234,567," we mean relative to this reference.

The reference is not any single person's genome. It is a composite assembly that the scientific community maintains and updates.

2.4 What is Variant Calling?

No two humans have identical genomes. The places where an individual's genome differs from the reference are called **variants**. The computational process of finding these differences is called **variant calling**.

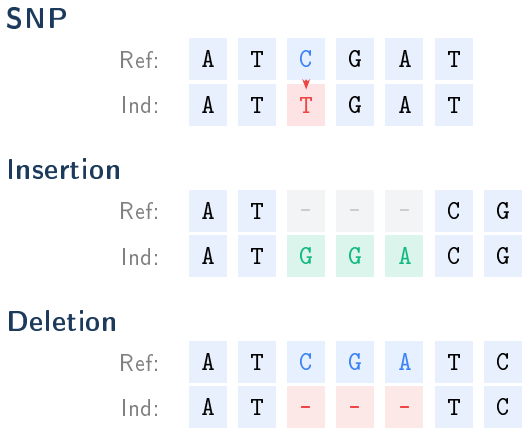

Figure 1: Three main types of small variants: SNPs change a single base; insertions add extra bases; deletions remove bases. Colored cells highlight the changed positions.

2.5 What is HG002?

HG002 (also known as NA24385) is a human genome sample from the Genome in a Bottle (GIAB) consortium. GIAB has very carefully characterized this sample — they know, with high confidence, exactly which variants are present. This makes it the gold standard for testing whether a variant calling pipeline is working correctly.

The GIAB consortium provides:

- A **truth VCF** — a file listing all known variants for HG002
- A **high-confidence BED** — a file defining the regions of the genome where the truth variants are reliable

2.6 The VCF File Format

VCF (Variant Call Format) is the standard file format for storing variant information. Here is a simplified example:

```
#CHROM POS     ID REF ALT QUAL FILTER INFO
chr1   1234567 .  A   G   50   PASS   DP=30
```

```
chr1    2345678 .   AT   A    45    PASS    DP=25
```

Each line represents one variant. The columns tell you which chromosome it is on (`CHROM`), the position (`POS`), what the reference base is (`REF`), what this individual has instead (`ALT`), a quality score (`QUAL`), and whether it passed quality filters (`FILTER`).

3 Background: The Infrastructure

3.1 What is an HPC Cluster?

A High-Performance Computing (HPC) cluster is a group of powerful computers (called **nodes**) connected by a fast network. You do not interact with these nodes directly. Instead, you log in to a **login node** and submit **jobs** — requests to run your programs. A **job scheduler** then decides which node has available resources and runs your job there.

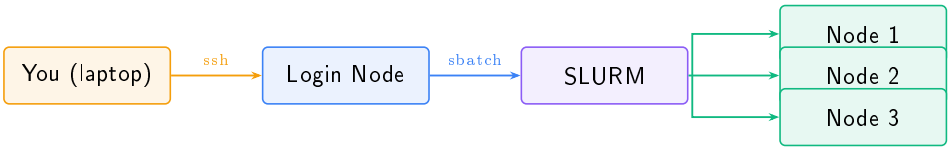

Figure 2: HPC cluster architecture: you connect via SSH to the login node, submit jobs via SLURM, and SLURM dispatches them to available compute nodes.

Variant calling on a full human genome requires 24+ GB of RAM, 8+ CPU cores, 50+ GB of disk space, and hours of continuous computation. A typical laptop would take days. An HPC node can finish in minutes to hours.

3.2 SLURM: The Job Scheduler

SLURM (Simple Linux Utility for Resource Management) is the most widely used job scheduler on HPC clusters. When you submit a job to SLURM, you tell it how many CPUs you need, how much memory, and how long you expect the job to take. SLURM puts your job in a **queue** and runs it when resources become available.

3.2.1 Key SLURM Commands

Command	What It Does
`sbatch script.slurm`	Submit a job script to the queue
`squeue -u $USER`	Show your currently queued/running jobs
`scancel <job_id>`	Cancel a specific job
`sinfo`	Show available partitions and their status
`sacct -j <job_id>`	Show detailed info about a completed job

Table 1: Essential SLURM commands

3.2.2 Anatomy of a SLURM Script

A SLURM script is a regular bash script with special `#SBATCH` directives at the top:

```
#!/bin/bash
#SBATCH -J my_job_name       # Job name (shows up in squeue)
#SBATCH -p gpu               # Partition (queue) to submit to
#SBATCH --cpus-per-task=8    # Number of CPU cores
#SBATCH --mem=24G            # Memory allocation
#SBATCH -t 03:30:00          # Max wall time (HH:MM:SS)
#SBATCH -o logs/job_%j.out   # Standard output file (%j = job ID)
#SBATCH -e logs/job_%j.err   # Standard error file

# Your actual commands go here
echo "Hello from the compute node!"
```

Example SLURM Script

Understanding SBATCH Directives

- `-J`: Just a label — makes it easier to identify your job in the queue.
- `-p`: The partition (think of it as a queue). Different partitions may have different hardware (e.g., GPU nodes vs CPU-only nodes).
- `-cpus-per-task`: How many CPU cores your job needs.
- `-mem`: How much RAM your job needs. If your job uses more than this, SLURM will kill it.
- `-t`: Maximum time. If your job runs longer, SLURM kills it. Always request a little more than you think you need.

3.3 Containers: Docker and Singularity

A **container** is a self-contained package that includes a program and all of its dependencies (libraries, configuration files, etc.). Think of it like a virtual machine, but much lighter and faster.

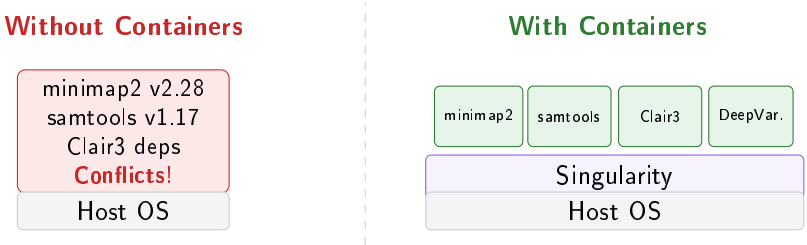

Figure 3: Containers isolate each tool with its own dependencies, eliminating version conflicts.

3.3.1 Docker vs Singularity

Feature	Docker	Singularity/Apptainer
Requires root access	Yes	No
Allowed on HPC clusters	Usually no	Yes (designed for HPC)
Can run Docker images	N/A	Yes (converts automatically)
Security model	Runs as root inside container	Runs as your user

Table 2: Docker vs Singularity comparison

Singularity was renamed to **Apptainer** in 2021 when it moved to the Linux Foundation. They are the same tool with the same commands. Many clusters still call it "singularity" while the binary might actually be `apptainer`. In our pipeline, we set `binary = 'apptainer'` in the Nextflow config for this reason.

A very common mistake: writing `singularity exec ...` inside a Nextflow process `script` block. If Nextflow is already running the process inside a container (via the `container` directive), then your manual call tries to run Singularity inside Singularity — which fails with `exit status 127` because the `singularity` binary does not exist inside the container. **Rule:** Always use the `container` directive. Never call `singularity exec` manually in the script block.

4 Nextflow: Complete Beginner's Guide

4.1 What is Nextflow?

Nextflow is a **workflow manager**. In bioinformatics, a typical analysis involves many steps that depend on each other — you need to align before you can sort, sort before you can call variants, etc. Nextflow lets you define these steps and their dependencies, and then it handles:

- **Execution order:** It figures out which steps can run in parallel and which must wait.
- **Error recovery:** If a step fails, you can fix the issue and resume from where it stopped (`-resume`).
- **HPC integration:** Each step can be automatically submitted as a separate SLURM job.
- **Container support:** Each step can run inside its own container.

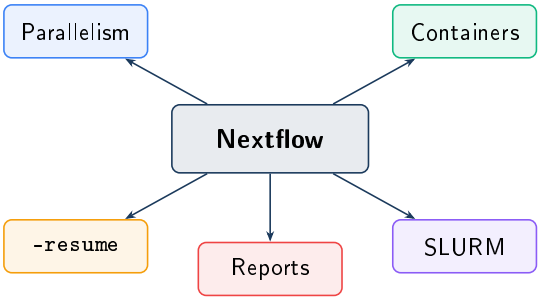

Figure 4: Nextflow handles five key concerns automatically: parallelism, containers, HPC submission, error recovery, and reporting.

4.2 DSL2: The Modern Nextflow Syntax

Nextflow has two syntax versions:

- **DSL1** (old): Everything is in one big script. Processes are implicitly connected.
- **DSL2** (current): Processes are defined separately and explicitly wired together in a `workflow` block. This is cleaner, reusable, and the only supported syntax going forward.

You enable DSL2 by putting this at the top of your `main.nf`:

```
nextflow.enable.dsl=2
```

Enabling DSL2

4.3 The Two Building Blocks

4.3.1 Processes

A **process** is a single computational step. It has `input`, `output`, `script`, and `container` blocks:

```
process align {
    tag "align"                 // Label for logs
    publishDir params.outdir, mode: 'copy' // Copy outputs here

    input:
    tuple path(ref), path(fai)  // Reference + its index
    path reads                  // FASTQ reads file

    output:
    path "hg002.sam"            // The output SAM file

    container "docker://quay.io/biocontainers/minimap2:2.28--he4a0461_0"

    cpus 8                      // Request 8 CPU cores
    memory '24 GB'              // Request 24 GB RAM
    time '3h'                   // Max 3 hours

    script:
    """
    minimap2 -t ${task.cpus} -ax map-hifi ${ref} ${reads} > hg002.sam
    """
}
```

Anatomy of a Nextflow Process

Key Things to Notice

- The script block uses triple quotes (`"""..."""`) so you can write multi-line bash.
- `${task.cpus}` is a Nextflow variable that resolves to whatever you set in `cpus`.
- `${ref}` and `${reads}` refer to the input file names.
- The `container` directive tells Nextflow which image to use — you never call `singularity exec` yourself.

4.3.2 Workflows

A **workflow** block wires processes together by passing the output of one process as the input to the next:

```
workflow {
    reads_ch = Channel.fromPath(params.reads, checkIfExists: true)
    ref_ch  = Channel.fromPath(params.ref, checkIfExists: true)

    ref_fai_ch = faidx(ref_ch)        // Step 1
    sam_ch = align(ref_fai_ch, reads_ch) // Step 2
    bam_tuple_ch = sort_index(sam_ch) // Step 3
    flagstat(bam_tuple_ch)            // Step 4

    // Steps 5a & 5b run in parallel!
    clair3_vcf_ch    = clair3(ref_fai_ch, bam_tuple_ch)
    deepvariant_vcf_ch = deepvariant(ref_fai_ch, bam_tuple_ch)

    happy_clair3(ref_fai_ch, clair3_vcf_ch)      // Step 6a
    happy_deepvariant(ref_fai_ch, deepvariant_vcf_ch) // Step 6b
}
```

The Workflow Block

In Nextflow, data flows between processes through **channels**. A channel is like a pipe — one process puts data in, another process takes data out.
`Channel.fromPath(params.reads)` creates a channel containing the file path to our FASTQ reads. When you write `sam_ch = align(ref_fai_ch, reads_ch)`, you are saying: "Take the data from `ref_fai_ch` and `reads_ch`, feed them into the `align` process, and put the output into `sam_ch`."
Parallelism happens automatically. Notice that `clair3()` and `deepvariant()` both take the same input (`ref_fai_ch` and `bam_tuple_ch`). Nextflow sees that these two processes do not depend on each other, so it runs them at the same time.

4.4 Tuples: Passing Multiple Files Together

Many bioinformatics tools need multiple related files together. For example, a sorted BAM file is useless without its index (`.bai`). In Nextflow, we pass them as a **tuple**:

```
// Output: two files bundled together
output:
tuple path("hg002.sorted.bam"), path("hg002.sorted.bam.bai")

// Input in the next process: destructure the tuple
input:
tuple path(bam), path(bai)
```

Tuple Example

4.5 Parameters and the publishDir Directive

Parameters let you configure the pipeline without editing the code. The `?:` operator provides a default value that users can override from the command line:

```
params.reads = params.reads ?: 'data/hg002_subset.fastq'
params.ref   = params.ref   ?: 'ref/hg38.fa'
params.outdir = params.outdir ?: 'results'
```

Parameter Definitions

The **publishDir** directive copies final output files from Nextflow's internal `work/` directory (hashed folders) to a human-readable location.

4.6 The -resume Flag

One of Nextflow's best features. If your pipeline fails halfway through, fix the problem and re-run with `-resume`. Nextflow will skip all the steps that already completed and only re-run the ones that failed.

4.7 Nextflow Configuration File

The `nextflow.config` file controls how the pipeline executes:

```
nextflow.enable.dsl=2       // Use DSL2 syntax

process {
  executor = 'slurm'         // Submit each process as a SLURM job
  queue    = 'gpu'            // Use the 'gpu' partition
  scratch = true              // Use local scratch disk on compute node
  cpus   = 2                  // Default: 2 CPUs per process
  memory = '4 GB'             // Default: 4 GB RAM per process
  time   = '30m'              // Default: 30 min per process
}

singularity {
  enabled     = true          // Enable Singularity containers
  autoMounts  = true          // Auto-mount file paths into containers
  cacheDir    = "${baseDir}/work/singularity_cache"
  binary      = 'apptainer'   // Use apptainer binary (modern name)
}

// Execution reports (generated automatically)
report  { enabled = true; file = "${baseDir}/results/nf_report.html" }
timeline { enabled = true; file = "${baseDir}/results/nf_timeline.html" }
```

```
trace   { enabled = true; file = "${baseDir}/results/nf_trace.txt" }
```

nextflow.config — Explained

The defaults in the `process { }` block (2 CPUs, 4 GB, 30 min) apply to all processes. But individual processes can override these — for example, the `align` process requests 8 CPUs and 24 GB because alignment is much more resource-intensive. Process-level settings always win over the config defaults.

5 The Pipeline: Step-by-Step Explanation

Now that we understand the theory, let us walk through every step of the pipeline in detail.

5.1 Pipeline Flow Diagram

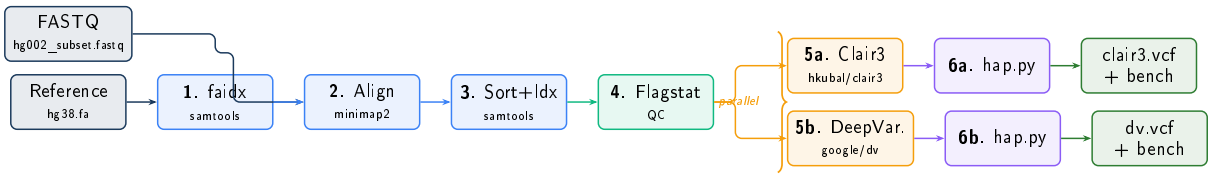

Figure 5: Complete pipeline flow: Steps 1–4 run sequentially. Steps 5a/5b (variant calling) and 6a/6b (benchmarking) run in parallel.

5.2 Step 1: Reference Indexing (faidx)

What and Why

Before any tool can quickly look up a specific region of the reference genome (e.g., "give me chromosome 7, positions 1000–2000"), it needs an **index**. The `.fai` index file is like a table of contents — it stores the name, length, and byte offset of each chromosome so tools can jump directly to any location.

The output is a **tuple** containing both the reference FASTA and its new index. This tuple is passed to all downstream processes that need the reference.

5.3 Step 2: Read Alignment (minimap2)

Alignment (also called mapping) is the process of finding where each sequencing read came from in the reference genome. minimap2 takes each read and finds the best matching location on hg38. The `-ax map-hifi` preset tells minimap2 to use parameters optimized for PacBio HiFi reads.

The output is a SAM (Sequence Alignment/Map) file — a text file where each line represents one read and where it aligned.

5.4 Step 3: SAM to BAM Conversion and Sorting

BAM is the compressed binary version of SAM. Converting SAM → BAM reduces file size significantly (our file went from ∼894 MB to ∼325 MB). **Sorting** arranges the reads by genomic position. After sorting, we also create a **BAM index** (`.bai`).

5.5 Step 4: Alignment QC (flagstat)

`samtools flagstat` gives a quick summary of alignment quality: how many reads were processed, how many mapped successfully, and what percentage mapped. In our run, we got a **99.58% mapping rate**, which is excellent for HiFi reads.

5.6 Step 5: Variant Calling

Two variant callers run **in parallel** on the same sorted BAM file:

Stage 1 — Pileup Model: A fast neural network scans every position in the genome and quickly identifies candidate variant sites by looking at the "pileup" — a column of all reads stacked at each position.
Stage 2 — Full-Alignment Model: A more accurate (but slower) neural network examines each candidate site in detail, looking at the full alignment context, and makes a final call with a quality score.
Model used: `/opt/models/hifi` (built-in PacBio HiFi model).

Step 1 — Make Examples: For each candidate position, DeepVariant creates a small image-like tensor from the pileup of reads. Each "pixel" encodes information like base quality and mapping quality.
Step 2 — Call Variants: A deep convolutional neural network (CNN) — similar to the ones used for recognizing photos — classifies each image as: homozygous reference (0/0), heterozygous variant (0/1), or homozygous variant (1/1).
Step 3 — Post-process: The raw predictions are refined and written out as a VCF file.
Model used: `-model_type=PACBIO` (built-in PacBio/HiFi mode).

5.7 Step 6: Benchmarking with hap.py

What and Why

hap.py (Haplotype Comparison Tool) compares called variants against the GIAB truth set and computes:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \qquad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- `-engine=vcfeval`: Haplotype-aware comparison (more accurate than simple position matching).
- `-pass-only`: Only evaluates variants with `PASS` in the `FILTER` column.
- `-f`: Restricts evaluation to GIAB high-confidence regions.

6 Benchmarking Results and Interpretation

6.1 The Raw Numbers

Table 3: SNP Performance — Clair3 vs DeepVariant

Metric	Clair3	DeepVariant
Truth Total	3,365,127	3,365,127
True Positives (TP)	23,708	13,589
False Negatives (FN)	3,341,419	3,351,538
False Positives (FP)	5,912	4,777
Genotype Errors (FP.gt)	4,891	4,732
Recall	0.70%	0.40%
Precision	80.05%	74.00%
F1 Score	1.40%	0.80%

Table 4: INDEL Performance — Clair3 vs DeepVariant

Metric	Clair3	DeepVariant
Truth Total	525,469	525,469
True Positives (TP)	1,941	1,257
False Negatives (FN)	523,528	524,212
False Positives (FP)	1,654	819
Genotype Errors (FP.gt)	703	565
Recall	0.37%	0.24%
Precision	54.33%	60.45%
F1 Score	0.73%	0.48%

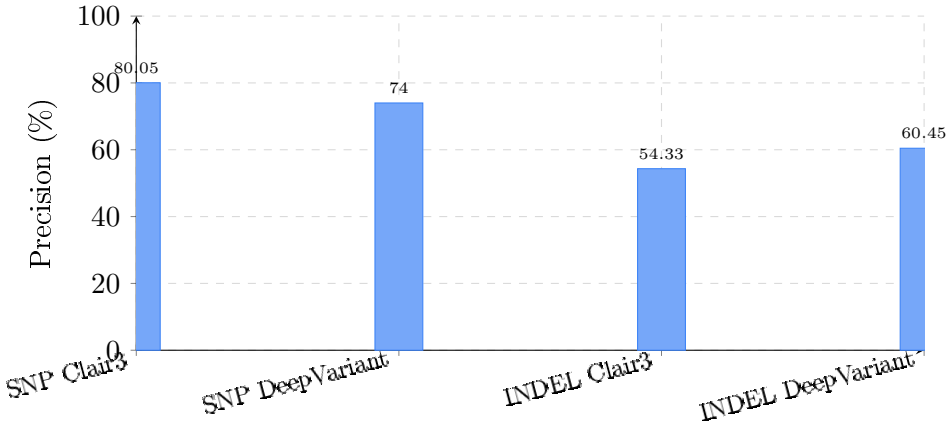

Figure 6: Precision comparison — both callers show strong precision for SNPs. DeepVariant has higher INDEL precision, while Clair3 has higher SNP precision.

6.2 Why is the Recall So Low?

The recall values (0.70% for SNPs, 0.37% for INDELs) look alarming, but they are completely expected given our experimental setup. We **subsampled the input data to 25%** of the original coverage (∼7–8x instead of ∼30x). At low coverage, many true variant sites simply do not have enough reads to be detected.
At full 30x coverage, both Clair3 and DeepVariant typically achieve >99% SNP recall on HG002.

6.3 What the Numbers Actually Tell Us

Precision is the more meaningful metric here. When the caller did make a call, how often was it correct?

- **Clair3 SNP precision: 80.05%** — 4 out of 5 SNP calls were correct
- **DeepVariant INDEL precision: 60.45%** — higher than Clair3's 54.33%
- **Clair3 was more sensitive:** It found 23,708 true SNPs vs DeepVariant's 13,589
- **Genotype errors dominate FPs:** Most "false positives" are actually the right variant at the right position, but with the wrong genotype (e.g., heterozygous instead of homozygous) — a direct consequence of low coverage

7 Practical: Reproducing the Pipeline

7.1 Prerequisites

Tool	Check Command	Notes
SLURM	`sinfo`	Should show available partitions
Singularity	`singularity -version`	Or `apptainer -version`
Nextflow	`nextflow -version`	Must be $\geq$ 25.x
8+ CPUs	`nproc`	On compute nodes
24+ GB RAM	`free -h`	On compute nodes
50+ GB disk	`df -h`	In your working directory

Table 5: Prerequisites checklist

7.2 Step-by-Step Commands

```
git clone https://github.com/cantcode27/HG002-Long-Read-Variant-Calling-Pipeline.git
cd HG002-Long-Read-Variant-Calling-Pipeline
```

Step 1: Clone the Repository

```
mkdir -p ref && cd ref
wget https://hgdownload.soe.ucsc.edu/goldenPath/hg38/bigZips/hg38.fa.gz
gunzip hg38.fa.gz && cd ..
```

Step 2: Download the Reference Genome

```
mkdir -p data
wget <HG002_PACBIO_HIFI_FASTQ_URL> -O data/hg002_full.fastq
seqtk sample -s100 data/hg002_full.fastq 0.25 > data/hg002_subset.fastq
```

Step 3: Download and Subsample Sequencing Data

```
mkdir -p benchmark && cd benchmark
GIAB="https://ftp-trace.ncbi.nlm.nih.gov/giab/ftp/release"
GIAB="${GIAB}/AshkenazimTrio/HG002_NA24385_son/latest/GRCh38"
wget ${GIAB}/HG002_GRCh38_1_22_v4.2.1_benchmark.vcf.gz
wget ${GIAB}/HG002_GRCh38_1_22_v4.2.1_benchmark.vcf.gz.tbi
wget ${GIAB}/HG002_GRCh38_1_22_v4.2.1_benchmark_noinconsistent.bed
cd ..
```

Step 4: Download the GIAB Truth Set

```
singularity pull docker://quay.io/biocontainers/minimap2:2.28--he4a0461_0
singularity pull docker://quay.io/biocontainers/samtools:1.17--h00cdaf9_0
singularity pull docker://hkubal/clair3
singularity pull docker://google/deepvariant:latest
singularity pull docker://jmcdani20/hap.py:v0.3.12
```

Step 5: Pull Containers

```
sbatch Scripts/run_nf.slurm     # Submit the pipeline
squeue -u $USER                  # Check job status
```

```
tail -f logs/nf_<job_id>.out     # Watch logs in real-time
```

Step 6: Run and Monitor

```
cat results/hg002.flagstat.txt              # Alignment quality
cat results/happy_clair3/happy.summary.csv  # Clair3 benchmark
cat results/happy_deepvariant/happy.summary.csv # DeepVariant benchmark
```

Step 7: Check Results

8 Common Errors and How to Fix Them

Error	Cause	Fix
Exit status 127	`singularity` not found inside container	Remove manual `singularity exec` calls; use the `container` directive
File self-copy	publishDir source = destination	Use separate output directory; use `mode: 'copy'`
SLURM fails immediately	Wrong partition name	Run `sinfo` to see available partitions; update `-p`
Out of memory	Insufficient `-mem`	Increase `memory` in the process definition
Time limit exceeded	Job took too long	Increase `time`; use `-resume`
Container not found	Wrong image name	Check exact name on Docker Hub or Quay.io
Report overwrite	Multiple Nextflow runs	Add `overwrite = true` to config

Table 6: Common errors and their fixes

When a process fails, Nextflow tells you its work directory. Go there and check `.command.sh` (the script), `.command.err` (errors), `.command.out` (output), and `.command.log` (Nextflow log). This is the fastest way to figure out what went wrong.

9 Tools Summary

Tool	Version	Container	Purpose
minimap2	2.28	`quay.io/.../minimap2:2.28...`	Read alignment
samtools	1.17	`quay.io/.../samtools:1.17...`	BAM processing
Clair3	latest	`hkubal/clair3`	Variant calling
DeepVariant	latest	`google/deepvariant:latest`	Variant calling
hap.py	v0.3.12	`jmcdani20/hap.py:v0.3.12`	Benchmarking
seqtk	—	conda install	FASTQ subsampling
Nextflow	25.10.4	Native install	Workflow orchestration

Table 7: Complete tools reference

10 Glossary

Term	Definition
BAM	Binary Alignment Map — compressed binary version of SAM
BAI	BAM Index — allows random access to specific regions of a BAM file
BED	Browser Extensible Data — file format defining genomic regions
Channel	Nextflow's data-passing mechanism between processes (like a pipe)
Container	A packaged environment with software and all dependencies included
Coverage	Average number of reads covering each position in the genome
DSL2	Domain Specific Language version 2 — the modern Nextflow syntax
FASTA	A text format for storing nucleotide sequences
FASTQ	Like FASTA but includes per-base quality scores
F1 Score	Harmonic mean of precision and recall
FN	False Negative — a true variant that the caller missed
FP	False Positive — a variant call that is not actually real
GIAB	Genome in a Bottle — consortium providing truth sets for benchmarking
GRCh38	The current standard human reference genome assembly (also called hg38)
hap.py	Haplotype comparison tool for variant call benchmarking
HG002	A GIAB reference sample (NA24385), the Ashkenazim son
HiFi	High Fidelity — PacBio's long-read technology with >99.9% accuracy
HPC	High-Performance Computing — a cluster of powerful computers
INDEL	Insertion or deletion variant (typically <50 bases)
Nextflow	A workflow manager for scientific computing pipelines
Precision	Of all variants called, what fraction are real? TP / (TP + FP)
Recall	Of all true variants, what fraction did we find? TP / (TP + FN)

Term	Definition
SAM	Sequence Alignment Map — text format for storing read alignments
Singularity	Container platform designed for HPC (now called Apptainer)
SLURM	Simple Linux Utility for Resource Management — HPC job scheduler
SNP	Single Nucleotide Polymorphism — a single-base variant
TP	True Positive — a variant call that matches the truth set
Tuple	In Nextflow, a group of related files passed together between processes
VCF	Variant Call Format — standard file format for genetic variants

Acknowledgments

This pipeline was developed as part of the Special Topics in Bioinformatics (BI-436) course at NUST SINES. The HG002 sequencing data is provided by the NHGRI Human Pangenome Reference Consortium, and the truth sets are maintained by the Genome in a Bottle consortium at NIST.

This document is for academic purposes only and is not intended for clinical diagnostic use.